

OS THREADS QUICK REFERENCE SHEET

MUTEX

Purpose: one thread at a time

Use for: shared variables, arrays, counters

Functions:

```
pthread_mutex_lock(&mutex;);
```

```
pthread_mutex_unlock(&mutex;);
```

Mental model: one key / one door

CONDITION VARIABLE

Purpose: thread waits for event/turn

Use for: alternating, producer-consumer, ordering

Functions:

```
pthread_cond_wait(&cond;, &mutex;);
```

```
pthread_cond_signal(&cond;);
```

```
pthread_cond_broadcast(&cond;);
```

Golden template:

lock → while(condition false) wait → work → signal → unlock

Mental model: sleep + wakeup

SEMAPHORE

Purpose: allow N threads simultaneously

Functions:

```
sem_init(&sem;,0,N);
```

```
sem_wait(&sem;);
```

```
sem_post(&sem;);
```

Mental model: parking lot with N spaces

BARRIER

Purpose: everybody waits for everybody

Functions:

```
pthread_barrier_init(&barrier;,NULL,n);
```

```
pthread_barrier_wait(&barrier;);
```

Mental model: race starting line

RWLOCK

Purpose: many readers, one writer

Reader:

```
pthread_rwlock_rdlock(&rwlock;);
```

Writer:

```
pthread_rwlock_wrlock(&rwlock;);
```

Unlock:

```
pthread_rwlock_unlock(&rwlock;);
```

JOIN

Purpose: wait for thread to finish

Function:

```
pthread_join(thread,NULL);
```

Difference:

join = one waits for one

barrier = everybody waits for everybody

FAST RECOGNITION

shared variable → mutex

turns / alternating → condition variable

all wait together → barrier

max N inside → semaphore

many readers → rwlock

MOST COMMON EXAM PATTERNS

1. local computation → lock → update global → unlock

2. while(not my turn) wait → work → signal

3. phase1 → barrier → phase2

4. sem_wait → work → sem_post